Parametri main()

Description	Uso dei parametri argc/argv di main()

```
int main(int argc, char *argv[])
```

- `int`: tipo di dato restituito dalla funzione indica se il programma è stato eseguito con successo (*return 0*) o se si è verificato un errore (return diverso da zero)
- `argc`: numero di argomenti passati al programma dalla riga di comando quando viene eseguito, ***incluso il nome del programma stesso***
- `argv`: array di stringhe che contiene gli argomenti passati al programma dalla riga di comando. argv[0] è il nome del programma, mentre argv[1] in poi contiene gli argomenti forniti dall'utente

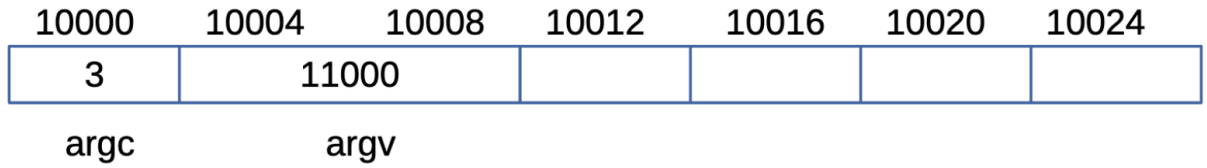
Ad esempio, se si esegue un programma da riga di comando come segue:

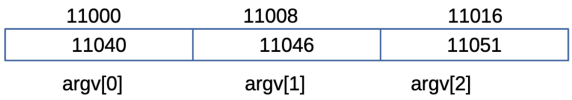
```
./mio_programma arg1 arg2 arg3
```

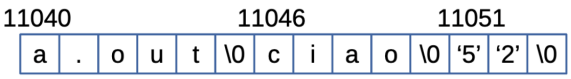
`argc` sarà 4 (poiché ci sono quattro argomenti, inclusivo del nome del programma), e argv sarà un array di stringhe che contiene il nome del programma e gli argomenti:

- `argv[0]` sarà una stringa contenente "./mio_programma"
- `argv[1]` sarà una stringa contenente "arg1"
- `argv[2]` sarà una stringa contenente "arg2"
- `argv[3]` sarà una stringa contenente "arg3"

❓ Questo meccanismo permette ai programmi di ricevere input dall'utente attraverso la riga di comando e di eseguire azioni specifiche in base agli argomenti forniti.








```
FILE *fopen(const char *filename, const char *mode)
```

Dove ***filename*** è un puntatore ad una stringa di caratteri che indica il nome del file da aprire, mentre *modo* indica invece la modalità con cui il file può essere aperto.

Ci sono diverse modalità disponibili per un file in apertura:

- `r` apre un file in ***lettura***.
- `r+` apre un file in **lettura/scrittura;** il file deve esistere.
- `w` crea un nuovo file in **scrittura**; se il file esiste viene cancellato il contenuto.
- `a` aggiunge (append) alla fine del file; se il file non esiste viene creato.
- `a+` aggiunge e legge a partire dalla fine del file; se il file non esiste viene creato.

`fscanf()`

```
int fscanf(FILE *fp, char *formato, [argomenti]...)
```

- **FILE** è un puntatore al file che vogliamo aprire
- **FORMATO** contiene uno più di questi elementi:
  - Caratteri diversi dagli spazi bianchi
  - Identificatori di formato.

la differenza con `scanf` è nel primo argomento che rappresenta il puntatore allo stream, mentre il significato degli altri argomenti rimane invariato.

```
int e = fscanf(file, "%d", &n);
```

Inoltre la funzione `fscanf`, a differenza di `scanf`, ritorna un valore intero, che rappresenta il numero di caratteri letti. Anche con `fscanf`, in caso di insuccesso la funzione restituisce una costante intera EOF (end of file).

La funzione `fscanf` non considera gli spazi all'interno del secondo argomento, quindi potrei anche scrivere `"%s%s%d,` ovvero tutto attaccato, senza modificare il risultato. Per lo stesso ragionamento potrei lasciare anche tanti spazi tra gli *specificatori* di formato senza in questo caso variare l'output.

> 📌 Quando legge il file la funzione `fscanf` percepisce gli argomenti dallo spazio che c'è tra di loro all'interno del file.

`fprintf()`

```
int fprintf(FILE *fp, char *formato, [argomenti]…)
```

- `FILE *fp` è il puntatore a *FILE* ottenuto dalla precedente chiamata fopen, che serve ad aprire il file

- la differenza con la funzione printf è proprio il primo argomento, rappresentato dal puntatore allo stream.
  Il significato degli altri argomenti passati alla funzione rimane invariato.

> 📌 `fprintf` restituisce un intero, cioè il numero di caratteri scritti nello stream.

```
fprintf(fp,"%s %s %d\n", cognome, nome, voto)
```